



Rich Clients for Web Services

Howard Rosen, Hans Muller, Scott Violet
Java Client Group
Sun Microsystems

Summary: Rich Clients + Web Services = Best Web Experience

Web services should offer something for everyone:

- Basic HTML clients for kiosks and surfers

- Rich clients for PCs, real users, business

Rich clients for web services are easier to build with Java™ technology

You can Web deploy clients with Java Web Start software



Overview

Background: web services, rich clients

Building rich clients

Deploying rich clients

Demo

Guide to client talks

Call to Action



Remember WORA?

Write Once, Run Anywhere™

It still matters

Solaris™ OS, Windows, Linux, HP-UX, AIX,
Tru64 UNIX, ...

... and once again: Apple!



Apple Demo



Last Year: “WebAwareWare”

Guerrilla clients for unsuspecting services

Dictionary, Language translation, etc.

Service characteristics

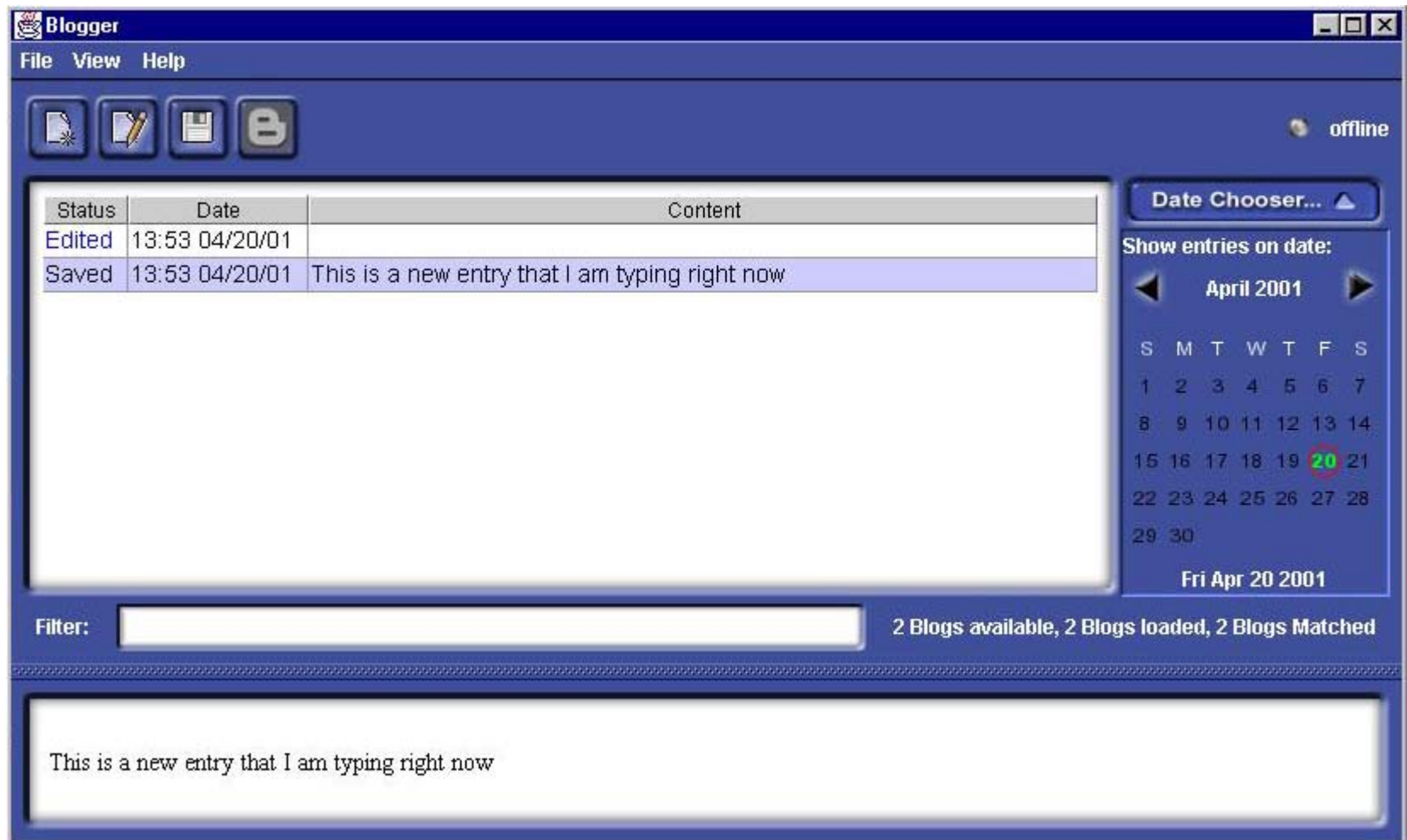
Variable bandwidth, high latency, HTTP

Poor error reporting, recovery

HTML data, screen scraping



Last Year: “WebAwareWare” BLOGGER Demo



This Year: Rich Clients for Consenting Servers

Basic client/server communication protocol
is still HTTP

Many higher level protocols emerging

- HTTP is just transport

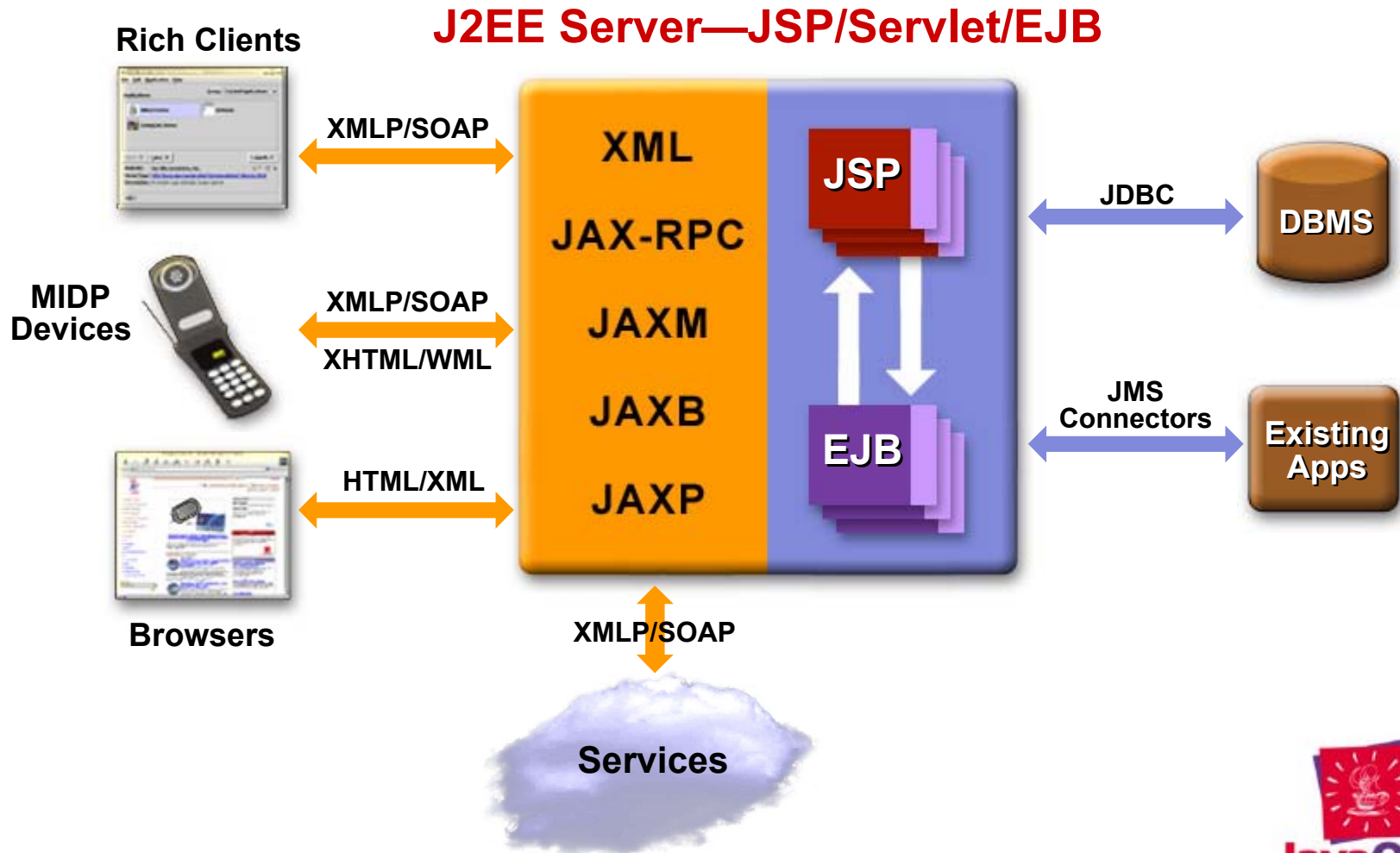
- XML encoding for messages

- Real error reporting/recovery

Good web services are not designed for a
specific client or client technology



Java™ Technology Web Services Model



Web Services

Web Services can provide leverage

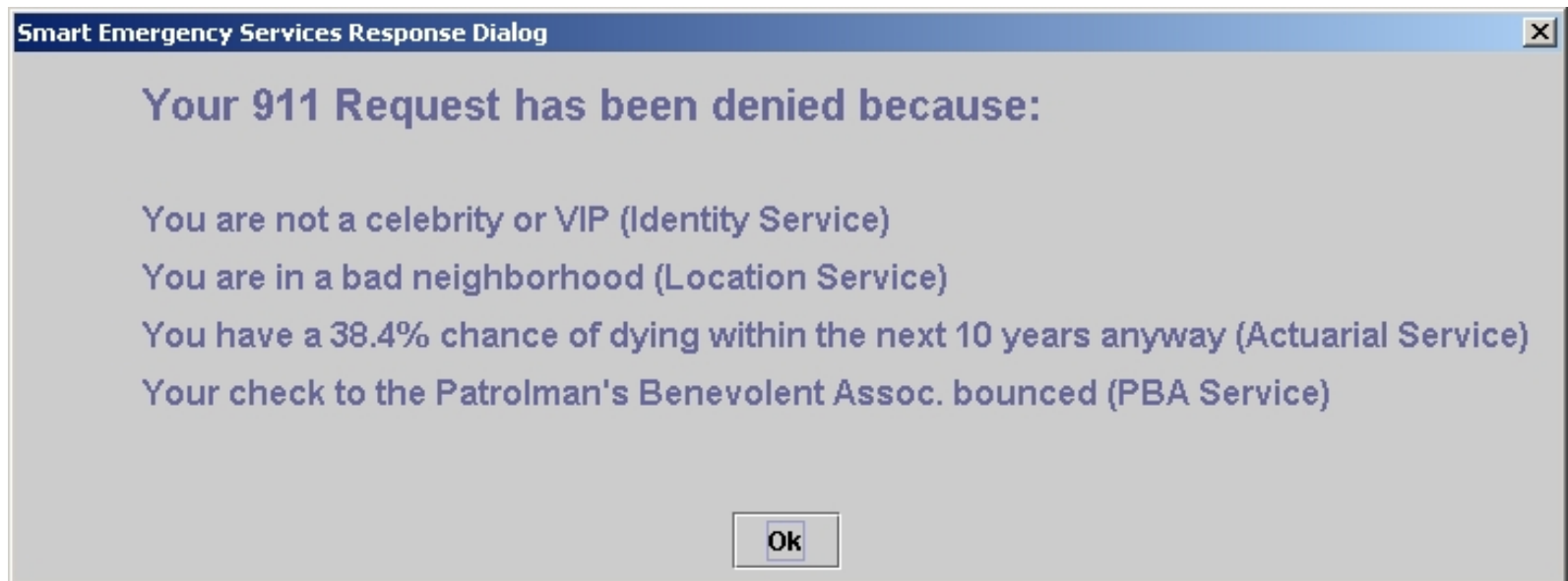
Context (identity, role, location, etc.)

Service to Service interactions benefit from “smarts”

Services shouldn't try to get smarter than the users—it can get ugly...



How Smart Is Smart Enough?



Why Build/Deploy Rich Clients

Dynamic, responsive, easy to use

Competitive edge / just another HTML page

Real-time visualization

Data sorting, filtering, direct manipulation

Decouple server performance from the GUI

Support offline usage

Because they're just better



Building Rich Clients

Mechanics of communicating with a service

- Using HTTP post, get

- Decoding XML responses

Rich Client Construction

- The N commandments for good GUIs

- Threads and work queues

Deployment



iPlanetTM Calendar and Directory Servers

Basis for our examples and demo

Directory Server support LDAP

Used by Calendar for user authentication,
preferences

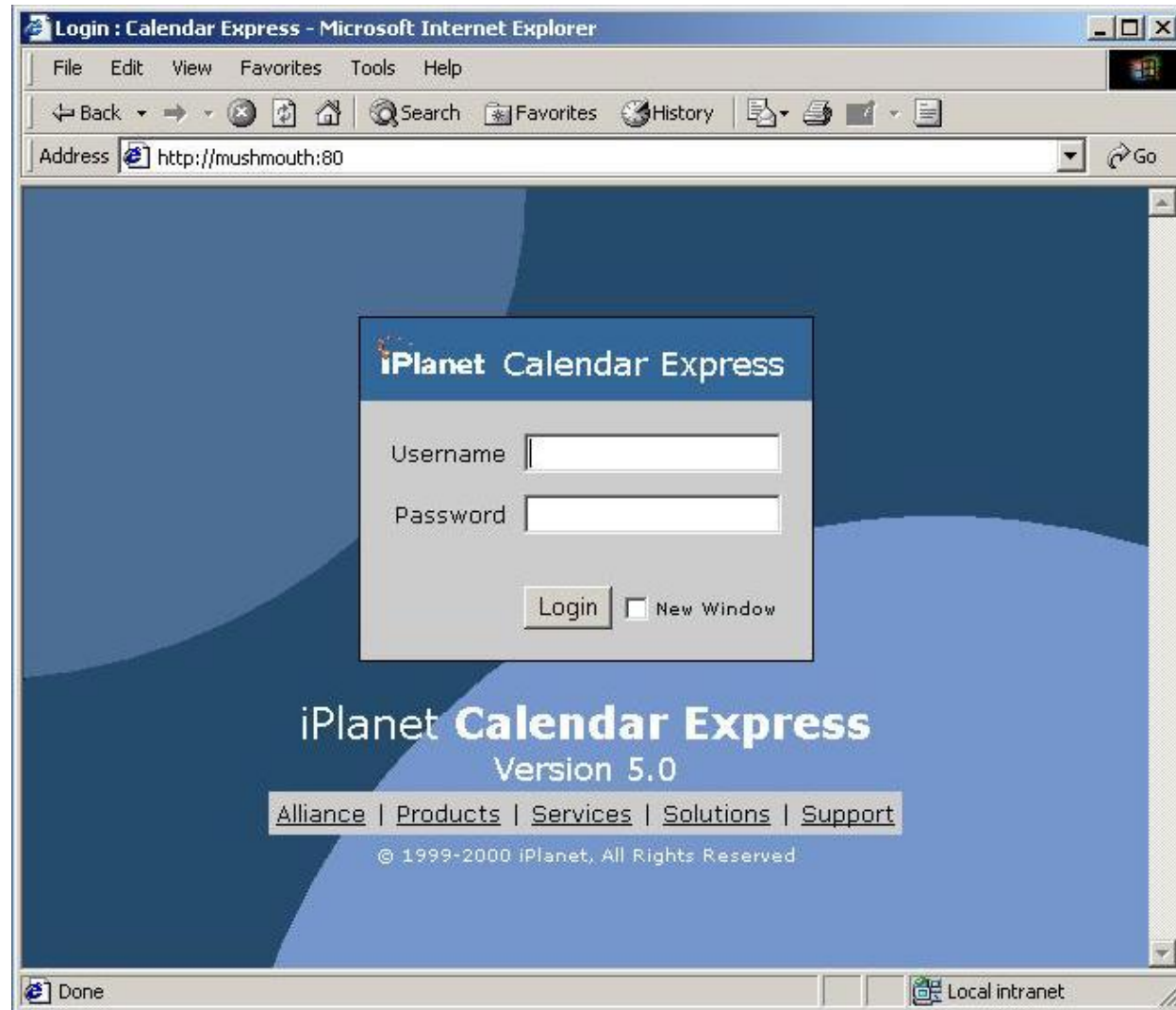
Calendar (5.0) protocol is WCAP

based on HTTP get/post

Responses are XML iCalendar format



Calendar HTML/JavaScript™ GUI Login



Calendar HTML/JavaScript GUI Month View

Calendar Express - iPlanet - Microsoft Internet Explorer

File Edit View Favorites Tools Help

Back Forward Stop Home Search Favorites History Print Mail

Address ng.sun.com/command.shtml?view=monthview&id=2mu9en0h8xeo38ub&date=20010316T000000&prevView=yearview&calid=calmaster&security=1 Go

iPlanet Calendar Express for cal master

View Calendars Groups Options Help Logout

New Event New Task Jump To Printable Search

Current Calendar: calmaster (calmaster)

March 2001

View: [overview](#) | [day](#) | [week](#) | [month](#)
[year](#) | [comparison](#)

Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
25	26	27	28	1	2	3
+	+	+	+	+	+	+
4	5	6	7	8	9	10
+	+	+	+	+	+	+
11	12	13	14	15	16	17
+	+	+	+	+	+	+
18	19	20	21	22	23	24
+	+	+	+	+	+	+
25	26	27	28	29	30	31
+	+	+	+	+	Overdue This is an example of a task 4:00pm - 5:00pm Smart Clients for Smart Services (Ugly Mug) Overdue Review iPlanet Calendar	+

America/Los_Angeles

March

S M T W T F S

25 26 27 28 1 2 3

4 5 6 7 8 9 10

11 12 13 14 15 16 17

18 19 20 21 22 23 24

25 26 27 28 29 30 31

Today is: 04/23/2001

Tasks

- No Invitations
- No Errors
- on cal's Calendar
- Tasks (2): 2 over due

Working with HTTP

Mechanics of communicating with a service

Using HTTP post, get

Decoding XML responses



Calendar Request: Login

URL to login to the Calendar server:

<http://host:port/login.wcap?user=jdoe&password=foo&fmt-out=text/xml>

Similar to a server "method call"

Method is *login.wcap*

Keyword instead of positional parameters

Values must be encoded

HTTP post form is similar



Calendar Login Response

```
<?xml version="1.0" encoding="UTF-8"?>
<iCalendar>
  <iCal version="2.0" prodid="-//iPlanet/Calendar Hosting Server//EN">
    <X-NSCP-WCAP-SESSION-ID>2mbp3b5tv65pb8uh</X-NSCP-WCAP-SESSION-ID>
    <X-NSCP-WCAP-USER-ID>jdoe</X-NSCP-WCAP-USER-ID>
    <X-NSCP-WCAP-CALENDAR-ID>jdoe</X-NSCP-WCAP-CALENDAR-ID>
    <X-NSCP-WCAP-ERRNO>0</X-NSCP-WCAP-ERRNO>
  </iCal>
</iCalendar>
```



Request Generation and Parsing— Create the URL

calenderServer.login(user);

```
void addParameter(StringBuffer sb, String p, Object o)
{
    sb.append(p);
    sb.append(URLEncoder.encode(o.toString()));
}
```

```
StringBuffer sb = new StringBuffer("/login.wcap");
addParameter(sb, "?user=", user.getName());
addParameter(sb, "&password=", user.getPassword());
addParameter(sb, "&fmt-out=", "text/xml");
```

```
URL url = new URL("http", host, port, sb.toString());
```



Request Generation and Parsing— Get the XML Document

```
calenderServer.login(user);
```

```
InputStream ist = url.openStream();
```

```
DocumentBuilderFactory dbf =  
    DocumentBuilderFactory.newInstance();
```

```
DocumentBuilder db = dbf.newDocumentBuilder();
```

```
Document doc = db.parse(ist);
```

```
ist.close();
```



Request Generation and Parsing— Extract Document Data

```
calenderServer.login(user);
```

```
String getBody(Node node) {  
    return node.getFirstChild().getNodeValue();  
}  
  
String getBody(Document doc, String tag) {  
    return getBody(doc.getElementsByTagName(tag).item(0));  
}  
  
user.setSessionID(getBody(doc, "X-NSCP-WCAP-SESSION-ID"));  
user.setCalendarID(getBody(doc, "X-NSCP-WCAP-CALENDAR-ID"));  
  
<X-NSCP-WCAP-SESSION-ID>2mbp3b5tv65pb8uh</X-NSCP-WCAP-SESSION-ID>  
<X-NSCP-WCAP-CALENDAR-ID>jdoe</X-NSCP-WCAP-CALENDAR-ID>
```

Server Requests and Responses Review

HTTP server requests are easy

`java.net.URL`

Many options for parsing responses

Rough and ready

DocumentBuilder to DOM
(`javax.xml.parsers`)

JAXB API



Building the Rich Client GUI

Focus on how to build a compelling client

Low bandwidth connection to the server

One second round trips typical

And users are not patient



Motivation: The N Commandments

Don't stare at the user

Let them know what's going on

Stay in sync with the data

If the application looks idle, it should be idle

You are not a graphics artist



Typical Rich Client GUI Task

Collect user input

First pass at validating the input

Server request based on input

Handle servers response

Errors: invalid input, runtime problems

Normal response, update GUI



Typical Rich Client GUI Task: Literal Implementation

```
if (!isLoginDataValid()) {  
    handleInvalidLogin();  
}  
else {  
    ServerResponse response = server.login(user);  
    if (response.isFailure()) {  
        handleLoginFailure(response);  
    }  
    else {  
        handleLoginSuccess(response);  
    }  
}
```



Literal Implementation: Fatal Flaw

One fatal flaw: server request is synchronous

Blocks GUI

User can't tell what's going on (no activity monitor)

User can't interrupt what's going on



Fatal Flaw: server.login() Blocks

```
if (!isLoginDataValid()) {  
    handleInvalidLogin();  
}  
else {  
    ServerResponse response = server.login(user);  
  
    if (response.isFailure()) {  
        handleLoginFailure(response);  
    }  
    else {  
        handleLoginSuccess(response);  
    }  
}
```



Using a Work Queue

`queue.enqueue(runnable3)`

Work Queue Thread

```
while(true) {  
    while(queue.isEmpty())  
        queue.wait();  
    queue.first().run();  
}
```

`runnable2`

`runnable1`

`runnable0.run();`

Building a Work Queue

```
public class workQueue {
    private final List queue = new LinkedList();
    private boolean stopped = false;

    public workQueue(String name) {
        new workerThread(name).start();
    }
    public final void enqueue(Runnable task) {
        synchronized(queue) {
            queue.add(task);
            queue.notify();
        }
    }
    public final void stop() {
        synchronized(queue) {
            stopped = true;
            queue.notify();
        }
    }
    // ...
}
```



The WorkQueue Thread

```
private class WorkerThread extends Thread {
    WorkerThread() {
        setPriority(Thread.MIN_PRIORITY);
    }
    public void run() {
        while (true) {
            Runnable task = null;
            synchronized(queue) {
                try {
                    while(queue.isEmpty() && !stopped)
                        queue.wait();
                }
                catch (InterruptedException e) {
                    stopped = true;
                }
                if (stopped) return;
                task = (Runnable)queue.remove(0);
            }
            task.run();
        }
    }
}
```



Server Request on the WorkQueue

```
if (!isLoginDataValid()) {  
    handleInvalidLogin();  
}  
else {  
    ServerResponse response = server.login(user);  
  
    if (response.isFailure()) {  
        handleLoginFailure(response);  
    }  
    else {  
        handleLoginSuccess(response);  
    }  
}
```



Server Request on the WorkQueue (Cont.)

```
ServerResponse response;
```

```
Runnable doHandleResponse = new Runnable() {  
    if (response.isFailure()) {  
        handleLoginFailure(response);  
    }  
    else {  
        handleLoginSuccess(response);  
    }  
};  
Runnable doRequestLogin = new Runnable() {  
    public void run() {  
        response = server.requestLogin();  
        EventQueue.invokeLater(doHandleResponse);  
    }  
};
```

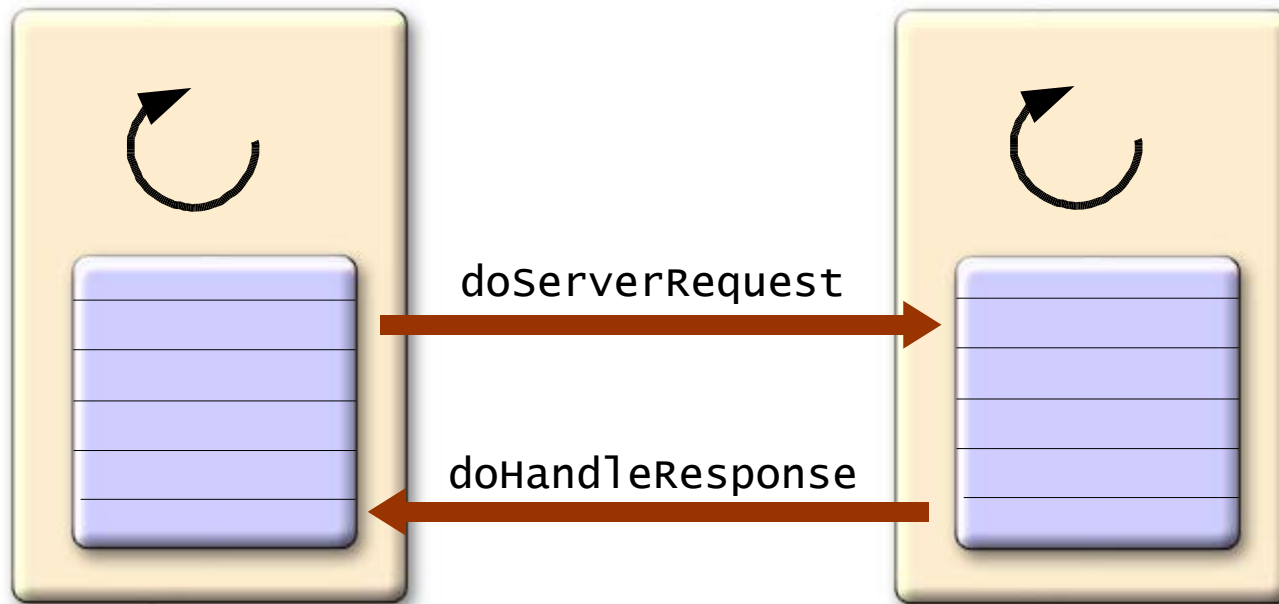
```
WorkQueue.enqueue(doRequestLogin);
```



Event Dispatching and Worker Queue Threads

Event Dispatching Thread

Work Queue Thread



Update GUI

Server Request/Response

Activity Monitor, Cancellation

Support for an activity monitor

Start activity monitor before server request begins

doHandleReponse is responsible for shutting the activity monitor off

Cancellation

Short circuit pending requests

Don't try and interrupt active requests



New Base Class Does the Work

Plumbing can be hoisted into a base class:

```
abstract class ServerAction implements ActionListener {  
    boolean isValid();  
    void serverRequest();  
    void handleResponse();  
}
```

isValid runs synchronously on the event dispatching thread

serverRequest runs asynchronously from the WorkQueue, and then:

handleResponse runs on the event dispatching thread



ServerAction in Action

```
class LoginAction extends ServerAction {
    LoginResponse response;
    boolean isDataValid() {
        return isLoginDataValid();
    }
    void serverRequest() {
        response = server.requestLogin();
    }
    void handleReponse() {
        if (response.isFailure()) {
            handleLoginFailure(response);
        }
        else {
            handleLoginSuccess(response);
        }
    }
}
```



Responsive GUI— Use a WorkQueue Review

Use a WorkQueue for server requests

Simple thread based primitive

Insulates GUI from tasks that may block

Serializes request processing

Communicates with the event dispatching thread via `EventQueue.invokeLater()`

Factor common GUI Action plumbing



Web Deploying a Rich Client Java Web Start software

Single click deployment:

- User clicks on a link in a browser

- Application starts

Eliminates the install step

Based on JSR-56: Java Network Launching Protocol and API



Web Deployment Basics

HTML File

```
<p>  
  To start the rich client application  
    <a href="client.jnlp">  
      please click here  
    </a>  
  Thanks.  
</p>
```



Web Deployment Basics

JNLP API File

```
<jnlp spec="1.0" codebase=...>
  <information>
    <title>My Rich Client App</title>
    <vendor>Sun Microsystems,
Inc.</vendor>
  </information>
  <resources>
    <j2se version="1.3"/>
    <jar href="client.jar"
version="1.0"/>
  </resources>
  <application-desc main-
class="Client"/>
</jnlp>
```



Web Deployment Review

It's easy!

On the web server

- Application jar files

- JNLP API file

- HTML file that contains JNLP file link

MIME type of the JNLP file:

- `application/x-java-jnlp-file`



Motivational Reminder

OK, that's enough plumbing details for today

Why go through the trouble? The answer is to build incredible Rich Client applications

You can't do this in HTML

But you can do it with the JFC/Swing API + AWT + Java 2D™ API + “Java i18n”!



Demo!



Java™ Technologies for the Desktop: Web to PC

This track is all things J2SE™ technology on the client: AWT, JFC, 2D, 3D, Media, Deployment, Apple Mac OS/X

For J2SE technology client developers, Hall B is your home base this week!

Key Sessions:

- What's New on the Client (TS-1309)

- Java Web Start or JSP™ components? (TS-1828)

- J2SE v1.3 release on Mac OS/X (TS-2813)

- The Java™ Image I/O API (TS-600)*



Call to Action

You have all the pieces—go do it!

Deploy with the J2SE 1.3.1 release and
Java Web Start software

Start developing with project “Merlin”
(J2SE 1.4 Beta release)

Don't let HTML rule the world





JavaOneSM
Sun's 2001 Worldwide Java Developer Conference™

Q&A



JavaOneSM

Sun's 2001 Worldwide Java Developer Conference*